Real-Time Expert Session Booking System

Multi-Agent Development & Execution Documentation

Project Overview

Objective

Build a real-time expert session booking platform using:

- React (Web)
- Node.js
- Express.js
- MongoDB
- Socket.io

The platform should allow users to:

- Browse experts
- Search and filter experts
- View available slots
- Book sessions
- View their bookings
- Receive real-time slot updates

Core Functional Requirements

1. Expert Listing Screen

Features:

- Display experts
- Search experts by name
- Filter experts by category
- Pagination
- Loading states
- Error states

---

2. Expert Detail Screen

Features:

- Show expert information
- Show slots grouped by date
- Real-time slot updates
- Disable booked slots

---

3. Booking Screen

Form Fields:

- Name
- Email
- Phone
- Date
- Time Slot
- Notes

Requirements:

- Form validation
- Success messages
- Prevent double booking

---

4. My Bookings Screen

Features:

- Fetch bookings by email
- Display booking status

Statuses:

- Pending
- Confirmed
- Completed

Backend Requirements

Required APIs:

- GET /experts
- GET /experts/:id
- POST /bookings
- PATCH /bookings/:id/status
- GET /bookings?email=

Critical Technical Requirements

Prevent Double Booking

Prevent:

- Same expert
- Same date
- Same slot

from being booked twice.

Use:

- MongoDB compound unique index

Real-Time Slot Updates

Use:

- Socket.io

Proper Error Handling

Requirements:

- Validation
- Meaningful API responses
- Centralized error middleware

Recommended Tech Stack

Frontend

- React + Vite
- React Router DOM
- Axios
- Socket.io Client
- Tailwind CSS
- React Hot Toast

Backend

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- Socket.io
- dotenv
- cors
- express-validator
- nodemon

Root Project Structure

```
expert-booking-system/
├── client/
├── server/
└── README.md
```

Backend Folder Structure

```
server/
├── config/
├── controllers/
├── middleware/
├── models/
├── routes/
```

```
├── socket/
├── seed/
├── server.js
└── .env
```

Frontend Folder Structure

```
client/
└── src/
    ├── api/
    ├── components/
    ├── hooks/
    ├── layouts/
    ├── pages/
    ├── socket/
    ├── utils/
    ├── App.jsx
    └── main.jsx
```

Database Design

Expert Model

```
{
  name: String,
  category: String,
  experience: Number,
  rating: Number,
  bio: String,
  availableSlots: [
    {
      date: String,
      slots: [String]
    }
  ]
}
```

Booking Model

```
{
  expertId: ObjectId,
  name: String,
  email: String,
  phone: String,
  date: String,
  timeSlot: String,
  notes: String,
  status: {
    type: String,
    enum: ["Pending", "Confirmed", "Completed"],
    default: "Pending"
  }
}
```

Critical Database Constraint

Compound Unique Index

```
bookingSchema.index(
  {
    expertId: 1,
    date: 1,
    timeSlot: 1
  },
  {
    unique: true
  }
);
```

Purpose:

- Prevent race conditions
- Prevent duplicate bookings
- Ensure data integrity

Environment Variables

Backend .env

```
PORT=5000
MONGO_URI=your_mongodb_uri
CLIENT_URL=http://localhost:5173
```

Frontend .env

```
VITE_API_URL=http://localhost:5000/api
```

API Documentation

1. GET /experts

Features

- Pagination
- Search
- Category filter

Query Params

```
?page=1
&limit=6
&search=rahul
&category=Fitness
```

Response

```
{
  "experts": [],
  "currentPage": 1,
```

```
  "totalPages": 3
}
```

---

2. GET /experts/:id

Returns:

- Expert details
- Available slots

---

3. POST /bookings

Responsibilities:

- Validate request
- Check slot availability
- Save booking
- Emit socket event
- Handle duplicate booking

---

4. PATCH /bookings/:id/status

Allowed Values:

- Pending
- Confirmed
- Completed

---

5. GET /bookings?email=

Returns bookings associated with user email.

Socket.io Real-Time Flow

Backend Event

```
io.emit("slotBooked", {
  expertId,
  date,
  timeSlot
});
```

Frontend Listener

```
socket.on("slotBooked", callback);
```

Frontend Pages

1. Expert Listing Page

Features:

- Expert cards
- Search bar
- Category filter
- Pagination
- Loading states
- Error handling

---

2. Expert Detail Page

Features:

- Expert information
- Available slots
- Real-time slot updates
- Disabled booked slots

3. Booking Page

Features:

- Booking form
- Validation
- API integration
- Success and error handling

---

4. My Bookings Page

Features:

- Search by email
- Booking history
- Status badges

Multi-Agent Architecture

```
Project Manager Agent
        │
        ├── Backend Architect Agent
        │       ├── Database Agent
        │       ├── API Agent
        │       ├── Validation Agent
        │       └── Socket.io Agent
        │
        ├── Frontend Architect Agent
        │       ├── UI Components Agent
        │       ├── Expert Listing Agent
        │       ├── Expert Detail Agent
        │       ├── Booking Agent
        │       └── My Bookings Agent
        │
        ├── QA & Testing Agent
        │
        └── Deployment & Documentation Agent
```

AGENT 1 — PROJECT MANAGER AGENT

Responsibilities

- Setup project structure
- Define coding standards
- Initialize Git repository
- Manage development sequence
- Configure environment variables

Deliverables

- Root folder structure
- Git initialization
- Project setup documentation

AGENT 2 — BACKEND ARCHITECT AGENT

Responsibilities

- Setup Express server
- Configure middleware
- Configure MongoDB connection
- Configure Socket.io
- Mount routes

Deliverables

- Express server setup
- Middleware configuration
- Socket server initialization

AGENT 3 — DATABASE DESIGN AGENT

Responsibilities

- Design schemas
- Create Mongoose models
- Implement indexes
- Seed dummy experts

Deliverables

- Expert model
- Booking model
- Compound unique index
- Seed data

AGENT 4 — API DEVELOPMENT AGENT

Responsibilities

Build APIs:

- GET /experts
- GET /experts/:id
- POST /bookings
- PATCH /bookings/:id/status
- GET /bookings?email=

Deliverables

- Controllers
- Route handlers
- Pagination logic
- Search and filter logic

AGENT 5 — VALIDATION & SECURITY AGENT

Responsibilities

- Request validation
- Error handling
- MongoDB error handling
- Centralized middleware

Deliverables

- Validation middleware
- Error middleware
- Standard API response format

AGENT 6 — SOCKET.IO REALTIME AGENT

Responsibilities

- Real-time updates
- Event emission
- Frontend listeners
- Slot synchronization

Deliverables

- Backend socket implementation
- Frontend socket integration
- Real-time slot updates

AGENT 7 — FRONTEND ARCHITECT AGENT

Responsibilities

- React project setup
- React Router setup
- Axios configuration
- Folder organization

Deliverables

- Frontend architecture
- Route configuration
- API layer

AGENT 8 — UI COMPONENTS AGENT

Responsibilities

Build reusable components:

- ExpertCard
- Loader
- ErrorMessage
- SearchBar
- Pagination

- Navbar
- SlotButton

Deliverables

- Responsive UI components
- Reusable frontend structure

AGENT 9 — EXPERT LISTING PAGE AGENT

Responsibilities

- Fetch experts
- Implement search
- Implement filters
- Implement pagination
- Render expert cards

Deliverables

- Homepage
- Pagination system
- Search/filter system

AGENT 10 — EXPERT DETAIL PAGE AGENT

Responsibilities

- Fetch single expert
- Render slot groups
- Implement realtime slot updates
- Disable booked slots

Deliverables

- Expert detail page
- Real-time slot UI

AGENT 11 — BOOKING PAGE AGENT

Responsibilities

- Build booking form
- Validate form
- Integrate booking API
- Handle success/error states

Deliverables

- Booking form page
- Validation system
- Booking submission flow

AGENT 12 — MY BOOKINGS PAGE AGENT

Responsibilities

- Fetch bookings by email
- Display booking history
- Render booking statuses

Deliverables

- My bookings page
- Booking status UI

AGENT 13 — QA & TESTING AGENT

Responsibilities

Test:

- APIs
- Realtime updates
- Double booking prevention
- Validation
- Pagination

Deliverables

- Tested APIs
- Bug fixes
- Edge case validation

AGENT 14 — DEPLOYMENT AGENT

Responsibilities

Deploy:

- Backend on Render
- Frontend on Vercel

Configure:

- Environment variables
- Production URLs

Deliverables

- Live frontend URL
- Live backend URL

AGENT 15 — DOCUMENTATION AGENT

Responsibilities

Prepare:

- README
- Setup guide
- Screenshots
- Demo video

Deliverables

- Complete documentation
- Submission-ready repository

Frontend Development Flow

Phase 1

- Setup React project
- Install dependencies
- Configure Tailwind CSS

Phase 2

- Create reusable components
- Setup routing
- Setup Axios

Phase 3

- Build Expert Listing page
- Build Expert Detail page

Phase 4

- Build Booking page
- Build My Bookings page

Phase 5

- Integrate Socket.io
- Add realtime updates

Phase 6

- Responsive UI
- Loading states
- Error handling

Backend Development Flow

Phase 1

- Setup Express server
- Configure MongoDB

Phase 2

- Create models
- Create routes
- Create controllers

Phase 3

- Add validation
- Add error middleware

Phase 4

- Implement Socket.io
- Handle realtime updates

Phase 5

- Seed database
- Test APIs

Testing Checklist

Backend Testing

- GET /experts works
- Pagination works
- Search works
- Filter works
- POST /bookings works
- Duplicate booking blocked
- PATCH status works
- GET bookings by email works

Frontend Testing

- Search updates UI
- Filters work correctly
- Pagination updates correctly
- Booking form validates
- Slot updates in realtime
- Loading states visible
- Error states visible

Deployment Plan

Backend Deployment

Platform:

- Render

Steps:

- Connect GitHub repo
- Add environment variables
- Deploy Node server

Frontend Deployment

Platform:

- Vercel

Steps:

- Connect GitHub repo
- Configure API URL
- Deploy React app

README Structure

README should include:

1. Project Overview
2. Features
3. Tech Stack
4. Installation Steps
5. Environment Variables
6. API Endpoints
7. Screenshots
8. Demo Video
9. Deployment Links

Demo Video Plan

Record:

1. Expert listing
2. Search & filter
3. Pagination
4. Booking process
5. Real-time slot update
6. Double booking prevention
7. My bookings page

Final Priority Order

```
1. Backend APIs
2. Database design
3. Compound unique index
4. Socket.io realtime updates
5. Frontend functionality
6. Validation and error handling
7. UI polishing
8. Deployment
9. Documentation
```

Most Important Interviewer Impression Points

1. Compound Unique Index

Shows backend maturity.

---

2. Socket.io Realtime Updates

Shows realtime architecture knowledge.

---

3. Proper Folder Structure

Shows professional organization.

---

4. Meaningful API Responses

Shows backend understanding.

---

5. Proper Loading/Error States

Shows frontend professionalism.

Final Execution Strategy

Focus on:

```
1. Functionality first
2. Realtime updates second
3. Error handling third
4. UI polish last
```

A clean working project with realtime updates and proper backend architecture is more valuable than an overdesigned UI.